

Alocação dinâmica de vetores e matrizes

Duas dimensões, dois caminhos: o vetor de ponteiros e o bloco único

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Um vetor dinâmico é um ponteiro *mais* um tamanho



Matriz como vetor de ponteiros: `double **m, m[i][j]`



Matriz como bloco único: um `malloc`, índice `i*nc + j`



Escolher entre as duas: contiguidade, custo, sintaxe



Os erros que o compilador não pega

O que a declaração fixa

```
double notas[50][4];
```

Fixa *duas* coisas antes de o programa rodar: quantos alunos cabem e quantas avaliações cada um tem.

Os dois modos de errar — agora ao quadrado

Pequeno demais: a turma tem 51 alunos, ou a disciplina passou a ter 5 provas.

Grande demais: `notas[1000][20]` reserva 160 KB para guardar 12 notas.

A pergunta de hoje

Na agenda, um `malloc` resolveu uma dimensão. E quando são duas?

O programa de hoje

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdint.h>
4
5  double **cria_matriz(int nl, int nc)
6  {
7      double **m = malloc(nl * sizeof(double *));
8      if (m == NULL)
9          return NULL;
10     for (int i = 0; i < nl; i++) {
11         m[i] = malloc(nc * sizeof(double));
12         if (m[i] == NULL) {                      /* falhou no meio do caminho */
13             for (int k = 0; k < i; k++)
14                 free(m[k]);
15             free(m);
16             return NULL;
17         }
18     }
19     return m;
20 }
```

```
23 void libera_matriz(double **m, int nl)
24 {
25     for (int i = 0; i < nl; i++)
26         free(m[i]);
27     free(m);
28 }
29
30 void medias_por_aluno(double **notas, int nl, int nc, double *saida)
31 {
32     for (int i = 0; i < nl; i++) {
33         double soma = 0.0;
34         for (int j = 0; j < nc; j++)
35             soma += notas[i][j];
36         saida[i] = soma / nc;
37     }
38 }
```

```
40  int main(void)
41  {
42      int  nalunos = 3, navaliacoes = 4;          /* viriam de um scanf */
43      double valores[] = { 8.0, 7.5,  6.0, 9.0,
44                          5.5, 6.0,  7.0, 4.5,
45                          9.5, 10.0, 8.5, 9.0 };
46
47      double **notas  = cria_matriz(nalunos, navaliacoes);
48      double *medias = malloc(nalunos * sizeof(double));
49      if (notas == NULL || medias == NULL) {
50          fprintf(stderr, "memoria insuficiente\n");
51          return 1;
52      }
```

```
54     for (int i = 0; i < nalunos; i++)
55         for (int j = 0; j < navaliacoes; j++)
56             notas[i][j] = valores[i * navaliacoes + j];
57
58     medias_por_aluno(notas, nalunos, navaliacoes, medias);
59     for (int i = 0; i < nalunos; i++)
60         printf("A: aluno %d -> media %.2f\n", i, medias[i]);
61
62     for (int i = 0; i < nalunos; i++) {
63         printf("B: linha %d começa em %p", i, (void *) notas[i]);
64         if (i > 0)
65             printf(" (%ld bytes depois da linha %d)",
66                    (long) ((uintptr_t) notas[i] - (uintptr_t) notas[i - 1]), i - 1);
67         printf("\n");
68     }
69     printf("C: uma linha ocupa %zu bytes\n", navaliacoes * sizeof(double));
70     printf("D: sizeof(notas) = %zu\n", sizeof(notas));
71     free(medias);
72     libera_matriz(notas, nalunos);
73     return 0;
74 }
```

Antes de compilar, escreva sua resposta

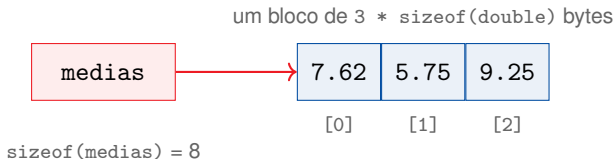
1. Quantas chamadas a `malloc` acontecem até a matriz ficar pronta, com 3 alunos e 4 avaliações?
2. Na linha **A**, qual é a média do aluno 0?
3. Na linha **B**, as três linhas saem encostadas umas nas outras na memória?
4. Na linha **D**, quanto vale `sizeof(notas)` — e por que não é $3 \times 4 \times 8$?

```
$ gcc -Wall -Wextra notas.c -o notas
$ ./notas
A: aluno 0 -> media 7.62
A: aluno 1 -> media 5.75
A: aluno 2 -> media 9.25
B: linha 0 começa em 0x600000b21200
B: linha 1 começa em 0x600000b21220 (32 bytes depois da linha 0)
B: linha 2 começa em 0x600000b21240 (32 bytes depois da linha 1)
C: uma linha ocupa 32 bytes
D: sizeof(notas) = 8
```

Os endereços mudam a cada execução — compare as *diferenças*, não os números.

Uma dimensão: o vetor dinâmico

Um vetor dinâmico é um ponteiro *mais* um número



O que o ponteiro guarda — e o que ele não guarda

Guarda o endereço da primeira casa. Não guarda quantas casas existem: `sizeof` devolve o tamanho do *ponteiro*, não o do bloco. Por isso `na` `unos` `viaja` `junto` `com` `medias` em toda função que o recebe.

Vetor de ponteiros

```
double **m;
```

Um bloco de ponteiros; cada ponteiro aponta para uma linha alocada separadamente.

1 + nl chamadas a `malloc`.

Bloco único

```
double *m;
```

Um bloco só, com as $nl \times nc$ casas em sequência; você calcula onde cada linha começa.

1 chamada a `malloc`.

As duas guardam a mesma matriz

A diferença não está no que elas representam, e sim em *como a memória é pedida* — e isso muda o custo, a sintaxe e o modo de liberar.

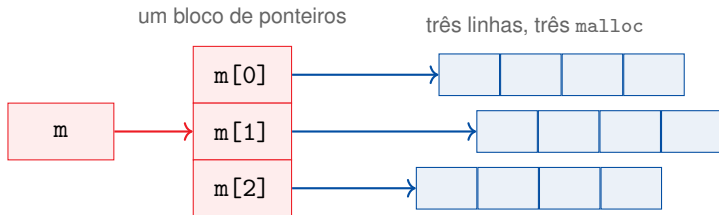
Matriz como vetor de ponteiros

```
double **m;
```

m aponta para um bloco de **ponteiros**. Cada $m[i]$ é um `double *` que aponta para um bloco de nc `doubles` — a linha i .

$m[i][j]$ **são dois saltos**

Primeiro o computador lê $m[i]$ (um endereço guardado no bloco de ponteiros); depois vai até esse endereço e lê a casa j . Dois acessos à memória para chegar a um número.



As três casas de ponteiros são vizinhas — vieram de um `malloc` só. As linhas, não: cada uma veio do seu próprio pedido e caiu onde o alocador quis.

```
double **m = malloc(nl * sizeof(double *));    /* 1: os ponteiros */
for (int i = 0; i < nl; i++)
    m[i] = malloc(nc * sizeof(double));        /* 2: cada linha    */
```

Duas contas diferentes

O primeiro malloc conta `sizeof(double *)` — ponteiros, 8 bytes cada. O segundo conta `sizeof(double)` — números, também 8 bytes aqui, mas por coincidência. Com char a diferença aparece: `sizeof(char *)` é 8 e `sizeof(char)` é 1.

Uma matriz 3×4 custa $1 + 3 = 4$ chamadas a malloc — e, depois, 4 chamadas a free.

A situação

O bloco de ponteiros foi alocado. As linhas 0 e 1 também. Aí o `malloc` da linha 2 devolve `NULL`.

A pergunta

O que `cria_matriz` deve fazer antes de devolver `NULL` para quem a chamou?

```
m[i] = malloc(nc * sizeof(double));  
if (m[i] == NULL) {  
    for (int k = 0; k < i; k++)    /* as linhas que já deram certo */  
        free(m[k]);  
    free(m);                      /* e o bloco de ponteiros      */  
    return NULL;  
}
```

Por que $k < i$ e não $k \leq i$

A linha i nunca chegou a existir: $m[i]$ é NULL. As anteriores existem e ficariam perdidas — ninguém mais tem o endereço delas depois que a função devolve NULL.

```
void libera_matriz(double **m, int nl)
{
    for (int i = 0; i < nl; i++)
        free(m[i]);      /* primeiro as linhas... */
    free(m);              /* ...depois os ponteiros */
}
```

A ordem não é estilo, é necessidade

Os endereços das linhas moram *dentro* do bloco de ponteiros. Se você liberar `m` primeiro, para ler `m[i]` depois você terá de ler memória que já devolveu — e os endereços das linhas se perdem junto.

O que a linha B mostrou

Cada linha ocupa 32 bytes, e as três saíram exatamente 32 bytes uma da outra: encostadas, como se fossem um bloco só.

A pergunta

Isso é uma garantia da linguagem, ou foi o que aconteceu *nesta* execução? Como você faria um experimento para decidir?

```
for (int i = 0; i < nl; i++) {  
    m[i] = malloc(nc * sizeof(double));      /* uma linha...      */  
    intruso[i] = malloc(nc * sizeof(double)); /* ...e outro pedido    */  
}
```

```
uma linha ocupa 32 bytes  
linha 0 em 0x600000f4d200  
linha 1 em 0x600000f4d240 (64 bytes depois da anterior)  
linha 2 em 0x600000f4d280 (64 bytes depois da anterior)
```

Quem decide onde cada bloco cai é o alocador, não você. Com `double **`, contiguidade entre linhas é sorte — nunca contrato.

```
int quantas[] = { 2, 4, 3 };           /* avaliacoes de cada aluno */  
for (int i = 0; i < nl; i++)  
    notas[i] = malloc(quantas[i] * sizeof(double));
```

```
aluno 0: 2 notas, media 5.50  
aluno 1: 4 notas, media 7.50  
aluno 2: 3 notas, media 8.00
```

Matriz irregular

Como cada linha tem seu próprio `malloc`, cada uma pode ter seu próprio tamanho. Em troca, o vetor `quantas` passa a viajar junto: sem ele, ninguém sabe onde cada linha termina.

O que aprendemos

- `double **m`: um bloco de ponteiros mais `n1` blocos de linha, $1 + n1$ chamadas a `malloc`;
- `m[i][j]` custa dois acessos: ler o ponteiro, depois ler o número;
- falha no meio do laço exige desfazer o que já deu certo;
- liberar é de dentro para fora: linhas antes do bloco de ponteiros;
- linhas vizinhas na memória são coincidência; linhas de tamanhos diferentes são de graça.

Matriz como bloco único

Definição: a matriz esticada em um bloco só

```
double *m = malloc(nl * nc * sizeof(double));
```

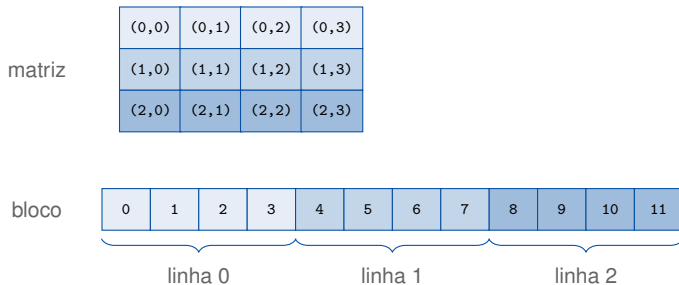
Um único bloco com todas as $nl \times nc$ casas em sequência. As linhas são guardadas uma depois da outra, da linha 0 à linha $nl - 1$.

O endereço da casa (i, j)

Pular i linhas inteiras custa $i \times nc$ casas; dentro da linha, andar mais j :

$$m[i * nc + j]$$

Repare que nc aparece na conta e nl não — por isso nc é o número que precisa viajar junto com o ponteiro.



A casa (1,2) da matriz é a casa $1 \times 4 + 2 = 6$ do bloco.

```
double *notas = malloc((size_t) nl * nc * sizeof(double));

for (int i = 0; i < nl; i++)
    for (int j = 0; j < nc; j++)
        notas[i * nc + j] = valores[i * nc + j];
...
free(notas);                                /* um free so */
```

```
linha 0 começa em 0x600002dec0c0  (0 bytes depois do início)
linha 1 começa em 0x600002dec0e0  (32 bytes depois do início)
linha 2 começa em 0x600002dec100  (64 bytes depois do início)
```

Agora as distâncias são exatas e previsíveis — e continuarão iguais em qualquer execução, em qualquer máquina.

O (size_t) não é decoração

```
malloc(nl * nc * sizeof(double));           /* nl * nc em int */  
malloc((size_t) nl * nc * sizeof(double));   /* nl * nc em size_t */
```

Onde a primeira linha quebra

Com $nl = nc = 50000$, o produto $nl * nc$ é calculado em `int` e estoura: $2,5 \times 10^9$ não cabe. O resultado vira um número negativo, que ao virar `size_t` vira um pedido gigantesco – ou pequeno demais, e o programa escreve fora do bloco.

Regra: converta *o primeiro* fator para `size_t` e deixe os outros seguirem.

```
void f(double **m, int nl, int nc);    /* vetor de ponteiros */
void g(double *m, int nl, int nc);    /* bloco unico          */
void h(double m[][4], int nl);        /* nc fixo na compilacao */
```

Por que `h` quase nunca serve

Para escrever `m[i][j]`, o compilador precisa saber `nc`. Em `h` ele sabe porque você escreveu 4 — e aí a função só funciona para matrizes de 4 colunas, que é justamente o que queríamos evitar.

Em `g`, quem calcula é você: `m[i * nc + j]`. O preço da liberdade é escrever a conta.

O que aprendemos

- um `malloc` e um `free`, sem laço e sem rollback;
- `m[i * nc + j]`: um acesso à memória, não dois;
- as linhas são contíguas por construção, não por sorte;
- `nc` viaja junto com o ponteiro em toda função;
- `(size_t)` no primeiro fator evita o estouro de `int` em matrizes grandes.

Escolher entre as duas

	double **m	double *m
Chamadas a malloc	$1 + nl$	1
Chamadas a free	$1 + nl$	1
Acesso	$m[i][j]$	$m[i*nc+j]$
Custo do acesso	dois saltos	um salto
Linhas contíguas	não garantido	sempre
Falha parcial	precisa desfazer	não existe
Linha nova	realloc nos ponteiros	realloc no bloco
Coluna nova	realloc em cada linha	remanejar tudo
Linhas irregulares	de graça	inviável

Nenhuma das duas é a certa. A pergunta é o que a sua matriz vai sofrer depois de criada.

Prefira `double **`

quando as linhas têm tamanhos diferentes, quando você precisa trocar linhas de lugar (basta trocar dois ponteiros) ou quando a legibilidade de `m[i][j]` pesa mais que tudo.

Prefira o bloco único

quando a matriz é grande e percorrida inteira, quando ela vai para uma biblioteca que espera memória contígua, ou quando o custo de alocar e liberar importa.

Por que a contiguidade importa

A memória é lida em pedaços: ao buscar um número, o computador traz os vizinhos junto. Num bloco único, percorrer `m[i][j]` na ordem `j` crescente aproveita esses vizinhos. Em linhas espalhadas, cada linha começa uma busca nova.

```
double **tmp = realloc(m, (nl + 1) * sizeof(double *));  
if (tmp == NULL) { /* m continua valido */ return 1; }  
m = tmp;  
  
m[nl] = malloc(nc * sizeof(double)); /* a linha nova, a parte */  
if (m[nl] == NULL) { return 1; }  
nl++;
```

Duas etapas, duas chances de falhar

O `realloc` pode mudar o bloco de ponteiros de lugar — daí o ponteiro temporário. As linhas antigas *não* se movem: quem se move é a lista de endereços.

No bloco único, basta um `realloc`: os dados antigos ficam nos mesmos índices.

Os erros que o compilador não pega

1. Liberar na ordem errada — `free(m)` antes das linhas

2. Trocar o índice — `m[j*nl+i]` no lugar de `m[i*nc+j]`

3. Andar com o ponteiro — e perder o começo do bloco

Dois deles o sanitizer acusa na hora. Um não — e é o mais perigoso.

Erro 1: liberar de fora para dentro



```
free(m);                                /* o bloco de ponteiros se foi... */  
for (int i = 0; i < nl; i++)  
    free(m[i]);                        /* ...e agora m[i] le memoria liberada */
```

```
==96988==ERROR: AddressSanitizer: heap-use-after-free on address 0x603000001b40  
READ of size 8 at 0x603000001b40 thread T0  
    #0 0x000104958885 in main ordem.c:14  
freed by thread T0 here:  
    #1 0x000104958842 in main ordem.c:12  
SUMMARY: AddressSanitizer: heap-use-after-free ordem.c:14 in main
```

Leia a mensagem inteira: ela diz *o que* aconteceu, *onde* o programa leu (linha 14) e *onde* aquela memória tinha sido liberada (linha 12).

```
m[i * nc + j] = 10 * i + j;  /* guardou certo */  
...  
printf("%5.0f", m[j * nl + i]);  /* leu errado */
```

0	3	12	21	esperado:	0	1	2	3
1	10	13	22		10	11	12	13
2	11	20	23		20	21	22	23

Por que nenhuma ferramenta reclama

Para qualquer i e j válidos, $j \cdot nl + i$ também cai *dentro* do bloco: o maior valor possível é $nl \times nc - 1$. Não há acesso inválido — só números no lugar errado. Quem detecta é você, conferindo a saída.

Erro 3: o ponteiro que andou



```
for (int i = 0; i < n; i++) {  
    *v = i;          /* escreve na posicao atual... */  
    v++;            /* ...e anda uma casa          */  
}  
free(v);            /* v nao aponta mais para o inicio do bloco */
```

```
==97042==ERROR: AddressSanitizer: attempting free on address which was not  
malloc()-ed: 0x603000001b60 in thread T0  
    #1 0x00010024a837 in main perdeu.c:15  
0x603000001b60 is located 0 bytes after 32-byte region [...]
```

free exige o endereço devolvido pelo malloc — nem um byte antes, nem um depois. Se precisar caminhar, caminhe com uma cópia e guarde o original.

```
$ gcc -Wall -Wextra -g -fsanitize=address prog.c -o prog && ./prog  
$ valgrind --leak-check=full ./prog
```

```
==97015==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x603000001b90  
WRITE of size 8 at 0x603000001b90 thread T0  
    #0 0x0001040dc8d8 in main estouro.c:14  
0x603000001b90 is located 0 bytes after 32-byte region
```

O que este achou

Um laço `for (j = 0; j <= nc; j++)` escrevendo uma casa além do fim da linha. Com `double **`, cada linha tem seu próprio fim — e o sanitizer conhece todos eles.

O que aprendemos

- libere de dentro para fora: linhas, depois o bloco de ponteiros;
- `i*nc+j` trocado não estoura nada — só devolve a matriz transposta, em silêncio;
- `free` só aceita o endereço exato que o `malloc` devolveu;
- `-fsanitize=address` custa um recompilar e devolve o arquivo e a linha;
- o que o sanitizer não vê, só a conferência da saída pega.

Mãos à obra

Tarefa 1

Acrescente `medias_por_avaliacao` — a média da coluna j , sobre todos os alunos. Repare em qual dos dois laços fica por fora e explique por que essa versão lê a memória "pulando".

Tarefa 2

Faça `nalunos` e `navaliacoes` virem de um `scanf` e as notas virem do teclado. Teste com 1×1 e com 0 alunos: o programa sobrevive aos dois?

Tarefa 3

Reescreva `notas.c` inteiro na forma de bloco único. Compare as duas versões: quantas linhas mudaram, quantos `free` sobraram?

A turma que muda de tamanho

Escreva `adicionar_aluno(double ***notas, int *nl, int nc)` que acrescenta uma linha zerada à matriz e atualiza `nl`.

Requisitos:

- use `realloc` no bloco de ponteiros com o padrão do ponteiro temporário — se falhar, a matriz antiga deve continuar utilizável;
- aloque a linha nova com `calloc`, e desfaça o `realloc` se ela falhar;
- escreva `remover_aluno` também, liberando a linha certa e fechando o buraco;
- o programa deve terminar sem vazamentos e sem erros do sanitizer.

O que mudou hoje

Na aula passada, o tamanho passou a ser decidido rodando. Hoje, as *duas* dimensões passaram — e apareceu uma escolha de projeto que não existia.

A escolha

Vetor de ponteiros ou bloco único não é questão de gosto: é decidir se você quer a sintaxe `m[i][j]` ou o controle sobre onde os números ficam.

Para levar

- ponteiro sozinho não guarda tamanho — `n1` e `nc` viajam junto;
- `double **`: $1 + n/$ blocos, linhas independentes, liberar de dentro para fora;
- bloco único: um `malloc`, `i*nc+j`, contiguidade garantida;
- o erro mais caro é o que não trava o programa.

-  André Backes.
Linguagem C: completa e descomplicada.
Elsevier, Rio de Janeiro, 2013.
-  Randal E. Bryant and David R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Pearson, Boston, 3 edition, 2015.
Seção 3.9: alinhamento de dados e layout de structs.
-  Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: teoria e prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.

-  **Paul Deitel and Harvey Deitel.**
C: como programar.
Pearson Prentice Hall, São Paulo, 6 edition, 2011.
-  **Ulrich Drepper.**
What Every Programmer Should Know About Memory.
Red Hat, Inc., 2007.
Seção 6.2: efeito da ordem de percurso de matrizes sobre a cache.
-  **Paulo Feofiloff.**
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.

-  **ISO/IEC.**
Iso/iec 9899:2018 — information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seção 6.7.2.1: structure and union specifiers.
-  **Brian W. Kernighan and Dennis M. Ritchie.**
C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.
-  **K. N. King.**
C Programming: A Modern Approach.
W. W. Norton, New York, 2 edition, 2008.
Capítulo 16: estruturas, uniões e enumerações.

-  Herbert Schildt.
C completo e total.
Makron Books, São Paulo, 3 edition, 1997.
-  Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.
Ferramenta usada no tutorial para detectar vazamentos e uso após free.
-  Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.

-  Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles.
Dynamic storage allocation: A survey and critical review.
In *International Workshop on Memory Management (IWMM)*, pages 1–116.
Springer, 1995.
Panorama de estratégias de alocação dinâmica e fragmentação.
-  Nivio Ziviani.
Projeto de algoritmos: com implementações em Pascal e C.
Cengage Learning, São Paulo, 3 edition, 2011.